A Decade of Slug

Eric Lengyel • March 17, 2026

What is now known as the **Slug Algorithm** for rendering fonts directly from Bézier curves on the GPU was developed in the Fall of 2016, so this year marks a full decade since its inception. I published a paper in JCGT about the technique in the middle of 2017, and my company sold the first license for version 1.0 of the Slug Library not long afterward. Since then, Slug has been licensed widely in the video games industry as well as by an array of companies specializing in areas like scientific visualization, CAD, video editing, medical equipment, and even planetariums. Our clients include Activision, Blizzard, id Software, 2K Games, Ubisoft, Warner Brothers, Insomniac, Zenimax, and Adobe among many others. Slug turned out to be the most successful software product I've ever made.

I originally created Slug in pursuit of better text rendering for the C4 Engine, where fonts needed to look great not only in the GUI, but inside game levels where they could appear very large and be viewed at oblique angles. Most recently, I used Slug to build the Radical Pie equation editor, which of course, needs extremely high-quality font rendering as well as vector graphics for things like brackets, radicals, and purely graphical items like arrows and highlights attached to mathematical expressions. Slug is also used to render the entire user interface inside the main editing window and all dialog boxes.

This post talks about what has changed within the rendering method since 2017, when the paper was published and the Slug Library was first released. It then concludes with an exciting announcement for those who may want to implement the Slug algorithm for their own projects.

Rendering Evolution

Slug renders text and vector graphics on the GPU directly from Bézier curve data without the use of texture maps containing precomputed or cached images of any kind. Doing this robustly, while also being fast and producing high quality results, is a difficult problem when we have to deal with floating-point round-off errors. Robustness requires that we never see artifacts like dropped pixels, sparkles, or streaks under any circumstances, provably so. Being fast means that the algorithm can render any reasonable amount of text on the game consoles of 2016 without impacting frame rates significantly. Producing high-quality results means that we get nicely antialiased text with smooth curves and sharp corners when viewed at any scale and from any perspective. The principles by which the Slug rendering algorithm achieves all of this are summarized in the following diagram. (Click for PDF version.)

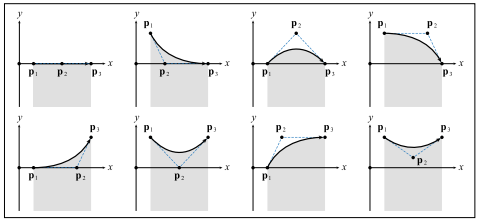
The Slug Algorithm

sluglibrary.com

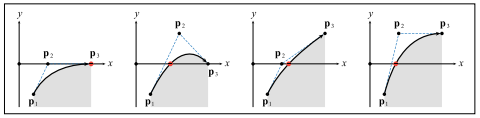
Root Eligibility

Class	$y_3 < 0$	$y_2 < 0$	$y_1 < 0$	Root 2	Root 1
A	0	0	0	0	0
B	0	0	1	1 ⊖	0
C	0	1	0	1 ⊖	1 ⊕
D	0	1	1	1 ⊖	0
E	1	0	0	0	1 ⊕
F	1	0	1	1 ⊖	1 ⊕
G	1	1	0	0	1 ⊕
H	1	1	1	0	0
				0x2E	0x74

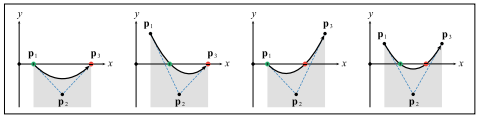
Class A



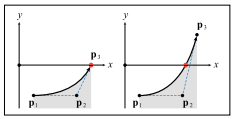
Class B



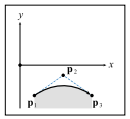
Class C



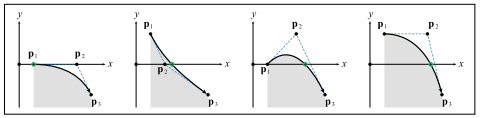
Class D



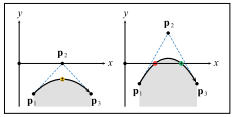
Class H



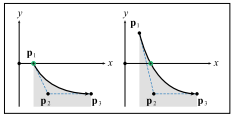
Class E



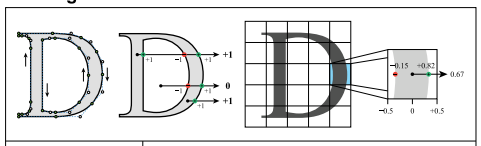
Class F



Class G

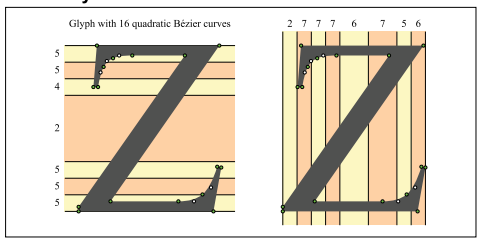


Winding Number

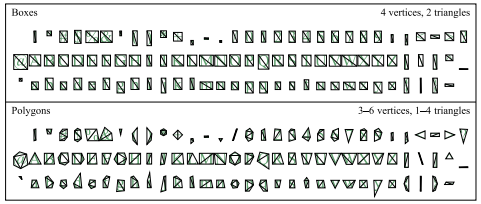


Quadratic Bézier curve	$\mathbf{p}(t) = (1-t)^2\mathbf{p}_1 + 2t(1-t)\mathbf{p}_2 + t^2\mathbf{p}_3 \qquad \mathbf{p}_i = (x_i, y_i)$	
Ray intersection equation	$p_y(t) = (y_1 - 2y_2 + y_3)t^2 - 2(y_1 - y_2)t + y_1 = 0$	
Potential solutions	$t_1 = \frac{b - \sqrt{b^2 - ac}}{a} \qquad t_2 = \frac{b + \sqrt{b^2 - ac}}{a} \qquad \frac{d}{dt}p_x(t_1) \le 0 \qquad \frac{d}{dt}p_x(t_2) \ge 0$ $a = y_1 - 2y_2 + y_3 \qquad b = y_1 - y_2 \qquad c = y_1$	
Change to winding number for ray in positive x direction	$+\,\mathrm{sat}\left(k p_x(t_1) + \frac{1}{2}\right)$ if root 1 eligible ⊕ $-\,\mathrm{sat}\left(k p_x(t_2) + \frac{1}{2}\right)$ if root 2 eligible ⊖	k = pixels per em
Change to winding number for ray in negative x direction	$-\,\mathrm{sat}\left(\frac{1}{2} - k p_x(t_1)\right)$ if root 1 eligible ⊖ $+\,\mathrm{sat}\left(\frac{1}{2} - k p_x(t_2)\right)$ if root 2 eligible ⊕	

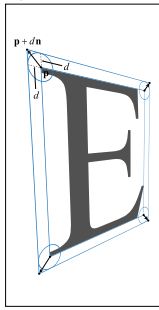
Geometry Bands



Bounding Polygons



Dynamic Dilation



$\mathbf{m}$ = transformation matrix from object space to clip space

w = viewport width $\qquad h$ = viewport height

d = object-space dilation distance

(p_x, p_y) = object-space vertex position

(n_x, n_y) = object-space vertex normal $\qquad \hat{\mathbf{n}} = \mathbf{n}/\|\mathbf{n}\|$

$$\Delta x = \frac{w}{2}\left[\frac{m_{00}(p_x + d\hat{n}_x) + m_{01}(p_y + d\hat{n}_y) + m_{03}}{m_{30}(p_x + d\hat{n}_x) + m_{31}(p_y + d\hat{n}_y) + m_{33}} - \frac{m_{00}p_x + m_{01}p_y + m_{03}}{m_{30}p_x + m_{31}p_y + m_{33}}\right]$$

$$\Delta y = \frac{h}{2}\left[\frac{m_{10}(p_x + d\hat{n}_x) + m_{11}(p_y + d\hat{n}_y) + m_{13}}{m_{30}(p_x + d\hat{n}_x) + m_{31}(p_y + d\hat{n}_y) + m_{33}} - \frac{m_{10}p_x + m_{11}p_y + m_{13}}{m_{30}p_x + m_{31}p_y + m_{33}}\right]$$

$$(\Delta x)^2 + (\Delta y)^2 = \frac{1}{4} \qquad \text{(half-pixel viewport-space dilation)}$$

$$s = m_{30}p_x + m_{31}p_y + m_{33} \qquad t = m_{30}\hat{n}_x + m_{31}\hat{n}_y$$

$$u = w\left[s\left(m_{00}\hat{n}_x + m_{01}\hat{n}_y\right) - t\left(m_{00}p_x + m_{01}p_y + m_{03}\right)\right]$$

$$v = h\left[s\left(m_{10}\hat{n}_x + m_{11}\hat{n}_y\right) - t\left(m_{10}p_x + m_{11}p_y + m_{13}\right)\right]$$

$$d = \frac{s^3 t + s^2\sqrt{u^2 + v^2}}{u^2 + v^2 - s^2 t^2}$$

Copyright © 2021

The method that determines root eligibility and calculates the winding number, which is responsible for robustness, is pretty much exactly the same now as it was in 2017 when Slug was first released. Some other parts of the rendering code that were described in the paper have changed over the years, however. I'll briefly describe the smaller changes here before talking about the big addition called "dynamic dilation" in its own section below.

The original paper included a description of a "band split optimization" that could be turned on when it was known that glyphs would be rendered at a large size. It did provide a speed increase for large glyphs, but it also introduced some divergence in the pixel shader that could hurt performance a little for text rendered at a small size. This optimization also required that the list of curves intersecting each band be stored twice, once sorted for rays pointing in one direction and again sorted for rays pointing in the opposite direction. The speed improvement was modest and didn't apply universally, so I decided to remove it. This eliminated some complexity in the pixel shader, and more importantly, it allowed the band data to be cut in half. The texture containing the band data now uses two 16-bit components instead of four.

In the Extensions section at the end of the paper, there was some discussion about supersampling. Though not necessary for rendering text at ordinary sizes, adaptive supersampling was implemented in early versions of Slug to enhance text drawn at very small sizes. If small text was rendered far away in a 3D scene, then supersampling reduced the amount of aliasing significantly as the camera moved, and because it was adaptive, the number of samples taken for larger text was still just one. Supersampling was removed because

(a) it made a difference only for text so small that it was barely readable anyway and (b) aliasing for tiny text was mitigated to a high degree by the dilation technique described below. Removing supersampling also simplified the pixel shader considerably. (Conditional compilation already eliminated the supersampling code when it was turned off, so its removal did not mean that the ordinary single-sample shader got any faster.)

The Extensions section also talked about adding a loop to the pixel shader in order to render multi-color emoji, which are essentially a stack of glyphs in which each layer has a different color. This proved to be unoptimal because many of the layers often only covered a small fraction of the total area of the composite glyph, but per-layer rendering calculations were still being performed over the full bounding polygon. It turned out to be better to render a bunch of independent glyphs on top of each other, even though it increased the amount of vertex data, so that each layer could have its own bounding polygon. This was faster, and it again simplified the pixel shader code.

Dynamic Dilation

There has been one major improvement to the rendering algorithm since the introduction of the Slug Library. It's called *dynamic dilation*, and it solves the problem discussed in a previous post from 2019 when it was first added to the code. Before dynamic dilation, the user had to manually specify a constant distance by which every glyph's bounding polygon would be expanded to ensure that all partially covered pixels get rasterized. This has two disadvantages: (a) if you choose a distance that's too small, then glyphs rendered

below a certain size start to have aliasing artifacts along their boundaries, and (b) any chosen distance will be too large for glyphs above a certain size, leaving empty space that eats up performance for no reason.

Dynamic dilation makes the optimal choice automatic, and it is recalculated in the vertex shader every time a glyph is rendered. The technique uses the current model-view-projection (MVP) matrix and viewport dimensions to determine how far a vertex needs to be moved outward along its normal direction in object space to effectively expand the bounding polygon by half a pixel in viewport space. This guarantees that the centers of any partially covered pixels are inside the bounding polygon so the rasterizer will pick them up. When text is viewed in perspective, the dilation distance can be different for each vertex. The code always produces the optimal value so that there's never any unnecessary padding that wastes GPU resources.

The dynamic dilation calculation done in the vertex shader is shown in the diagram above, but I haven't provided a derivation of it anywhere. So here we go. The goal is to find the distance d we must move an object-space vertex position $\mathbf{p} = (p_x, p_y, 0, 1)$ along its normal vector $\mathbf{n} = (n_x, n_y, 0, 0)$ for it to correspond to a half-pixel expansion of the bounding polygon in viewport space. The normal does not have unit length, but is instead scaled so that it would point to the new vertex location if both adjacent sides of the bounding polygon were to be pushed outward by one unit of distance, as shown in the diagram. We first calculate the distance d along the unit normal direction $\hat{\mathbf{n}} = (\hat{n}_x, \hat{n}_y, 0)$ and then apply that to the original normal vector $\mathbf{n}$ to obtain the new vertex position $\mathbf{p} + d\mathbf{n}$.

By applying the MVP matrix **m** (which is 4×4), the perspective divide, and the viewport scaling by its width *w* and height *h* to an object-space position **p** offset by the distance *d* in the unit normal direction $\hat{\mathbf{n}}$, we can express differences Δx and Δy in viewport space as

$$\Delta x = \frac{w}{2}\left[\frac{m_{00}(p_x + d\hat{n}_x) + m_{01}(p_y + d\hat{n}_y) + m_{03}}{m_{30}(p_x + d\hat{n}_x) + m_{31}(p_y + d\hat{n}_y) + m_{33}} - \frac{m_{00}p_x + m_{01}p_y + m_{03}}{m_{30}p_x + m_{31}p_y + m_{33}}\right]$$
$$\Delta y = \frac{h}{2}\left[\frac{m_{10}(p_x + d\hat{n}_x) + m_{11}(p_y + d\hat{n}_y) + m_{13}}{m_{30}(p_x + d\hat{n}_x) + m_{31}(p_y + d\hat{n}_y) + m_{33}} - \frac{m_{10}p_x + m_{11}p_y + m_{13}}{m_{30}p_x + m_{31}p_y + m_{33}}\right].$$

If we set $(\Delta x)^2 + (\Delta y)^2 = (\frac{1}{2})^2$, then the offset in viewport space is one-half pixel. We just need to solve this equation for *d*, but it gets pretty messy. When we multiply everything out, simplify as much as possible, and write this as a quadratic equation in *d*, we get

$$\begin{aligned}&\big[w^2(m_{03}(m_{30}\hat{n}_x + m_{31}\hat{n}_y) - m_{33}(m_{00}\hat{n}_x + m_{01}\hat{n}_y) + (m_{00}m_{31} - m_{01}m_{30})(p_x\hat{n}_y - p_y\hat{n}_x))^2\\&+ h^2(m_{13}(m_{30}\hat{n}_x + m_{31}\hat{n}_y) - m_{33}(m_{10}\hat{n}_x + m_{11}\hat{n}_y) + (m_{10}m_{31} - m_{11}m_{30})(p_x\hat{n}_y - p_y\hat{n}_x))^2\\&- (m_{30}p_x + m_{31}p_y + m_{33})^2(m_{30}\hat{n}_x + m_{31}\hat{n}_y)^2\big]d^2 - 2\big[(m_{30}p_x + m_{31}p_y + m_{33})^3(m_{30}\hat{n}_x + m_{31}\hat{n}_y)\big]d\\&- (m_{30}p_x + m_{31}p_y + m_{33})^4 = 0.\end{aligned}$$

It is convenient to make the assignments $s = m_{30}p_x + m_{31}p_y + m_{33}$ and $t = m_{30}\hat{n}_x + m_{31}\hat{n}_y$, which let us write

$$\begin{aligned}&\big[w^2(s(m_{00}\hat{n}_x + m_{01}\hat{n}_y) - t(m_{00}p_x + m_{01}p_y + m_{03}))^2\\&+ h^2(s(m_{10}\hat{n}_x + m_{11}\hat{n}_y) - t(m_{10}p_x + m_{11}p_y + m_{13}))^2 - s^2t^2\big]d^2\\&- 2s^3td - s^4 = 0.\end{aligned}$$

Further assigning

$$\begin{aligned}u &= w(s(m_{00}\hat{n}_x + m_{01}\hat{n}_y) - t(m_{00}p_x + m_{01}p_y + m_{03}))\\v &= h(s(m_{10}\hat{n}_x + m_{11}\hat{n}_y) - t(m_{10}p_x + m_{11}p_y + m_{13}))\end{aligned}$$

finally gives us the simplified quadratic equation

$$(u^2 + v^2 - s^2t^2)d^2 - 2s^3td - s^4 = 0,$$

which has the solutions

$$d = \frac{s^3t \pm s^2\sqrt{u^2 + v^2}}{u^2 + v^2 - s^2t^2}.$$

Choosing the plus sign obtains the distance outward along the unit normal vector that the vertex needs to be moved for a half-pixel dilation. To make sure the glyph is still rendered at the original size, the em-space sampling coordinates also need to be offset. A 2×2 inverse Jacobian matrix is stored with each vertex, and it gives us the information we need to transform an object-space displacement into an em-space offset vector.

The Jacobian matrix, before inverting, is the upper-left 2×2 portion of the transformation matrix that converts em-space coordinates to object-space coordinates, accounting for scale, stretch, skew, and possibly flips of the coordinate axes.

Patent Announcement

I was granted a patent for the Slug algorithm in 2019, and I legally have exclusive rights to it until the year 2038. But I think that's too long. The patent has already served its purpose well, and I believe that holding on to it any longer benefits nobody. Therefore, effective today, I am permanently and irrevocably dedicating the Slug patent to the public domain. That means anybody can freely implement the Slug algorithm from this day forward without a license for whatever purpose they want, and they don't need to worry about infringing upon any intellectual property rights. (For any legal experts reading this, my company has filed form SB/43 with the USPTO and paid the fee to disclaim the terminal part of the term for patent #10,373,352, effective March 17, 2026.)

To aid in implementations of the Slug algorithm, reference vertex and pixel shaders based on the actual code used in the Slug Library have been posted in a new GitHub repository and made available under the MIT license. The pixel shader is a significant upgrade compared to the code included with the JCGT paper, and the vertex shader includes dynamic dilation, which had not yet been implemented when the paper was published.

See Also

- *GPU-Centered Font Rendering Directly from Glyph Outlines*, Journal of Computer Graphics Techniques, 2017.
- Slug algorithm reference shaders on GitHub.

- Original paper presentation, i3D Symposium, 2018.
- “GPU Font Rendering: Current State of the Art” presentation, 2018.
- “Dynamic Glyph Dilation” blog post, 2019.
- Slug Library website.

Eric Lengyel’s Blog • **Copyright © 2026**